



# ***Licenciatura em Telecomunicações e Informática***

## ***Arquitetura e Tecnologia de Computadores o microcontrolador 8051***

**Paulo Cardoso**

Grupo de Sistemas Embebidos  
Departamento de Eletrónica Industrial  
Escola de Engenharia  
Universidade do Minho





# Sumário

- Microcontroladores
- O 8051
- Arquitetura
- O *core* do microcontrolador
- Organização da memória
- O conjunto de instruções



- Microcontroladores
  - Resposta a uma necessidade da indústria
    - Sistemas computacionais para ambientes de controlo
      - Onde para além de um “computador” é necessário
        - I/O digital e analógico
        - Temporizadores
        - PWM (Pulse Width Modulation)
        - Comunicação série (vários protocolos)
  - Por outro lado os ambientes industriais são agressivos
    - Condições ambientais, ruído eletromagnético
      - Fazer um computador com processador, memória e os circuitos integrados (CI) para implementar as funcionalidades acima
        - Solução mais sujeita a falhas e mais cara



- Microcontroladores
  - Resposta a uma necessidade da indústria
    - A solução é um CI único
      - Onde processador e periféricos estão integrados
        - Não apenas empacotados em conjunto no mesmo CI mas arquiteturalmente integrados
          - I.e. o processador “conhece” os periféricos
  - Assim, quando se fala de um microcontrolador
    - Fala-se de um sistema que integra
      - Processador, memória
      - Pinos de I/O digital e analógico (com DACs/ADCs)
      - *Timers*, PWM
      - Comunicação série básica e uma variedade de protocolos tais como I2C, SPI, CAN



# *O microcontrolador 8051*

## *o 8051*

- O 8051
  - Microcontrolador de 8 bits lançado em 1980
    - O QUÊ??????
      - Hoje em dia o mundo é dominado por microcontroladores ARM de 16 e 32 bits
    - Ainda muito usado atualmente
      - A patente expirou
        - Vários fabricantes introduziram novas versões do 8051
          - Mantendo a compatibilidade original
          - Usando as últimas técnicas de desenho de processadores
            - Aumentando significativamente o desempenho
            - E.g.: O Intel 8051 original executava a 12MHz e usava 12 ciclos para executar 1 instrução (1MHz, 1MIP- milhões de instruções por segundo)
            - O nosso micro executa a 100MHz a 100 MIPS (100x mais rápido que o 8051 clássico uma frequência apenas cerca de 8x mais)
          - Introduzindo novas características



# *O microcontrolador 8051*

## *o 8051*

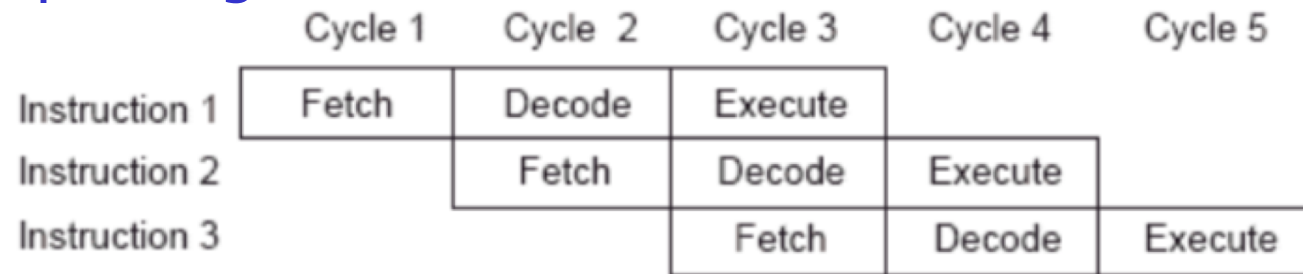
- O 8051
  - Ainda muito usado atualmente
    - Uma base instalada de milhões de linhas de código
    - Para além disso tem algumas vantagens arquiteturais
      - Implementa a capacidade de realizar operações lógicas diretamente em alguns registos e locais de memória
        - Este é um recurso muito útil em quase todas as aplicações de controlo, sejam elas industriais ou outras
          - Essas operações podem ser realizadas num único ciclo, permitindo um melhor tempo de resposta de controlo
    - Os portos de I/O são acedidos à velocidade do processador
      - Noutras arquiteturas estão separados por um barramento com desempenho inferior



# *O microcontrolador 8051 o 8051*

- Os 8051 atuais
  - Algumas novidades de implementação

- Pipelining



- MAC (multiply–accumulate)
    - Operações usadas em processamento digital de sinal
  - Melhor sistema de interrupções
    - Tempo de resposta (latência) melhorado
    - Mecanismos mais sofisticados de prioridades
  - *Debug* por hardware (*in-system debugging*)



# *O microcontrolador 8051 o 8051*

- Os 8051 atuais
  - Vantagens do ponto de vista comercial
    - Menor consumo que os micros mais poderosos
    - Custo mais baixo
    - Atrativos para soluções menos exigentes
      - E para soluções *low cost*
    - Apelativo para muitas soluções IoT
      - Onde o *sensing* é mais importante que o *processing*





# *O microcontrolador 8051 o 8051*

- Os 8051 atuais
  - Vantagens de programação
    - Debug por hardware
      - Tal como os os micros mais poderosos
    - Mais fácil de programar
      - Dado ter complexidade mais baixa



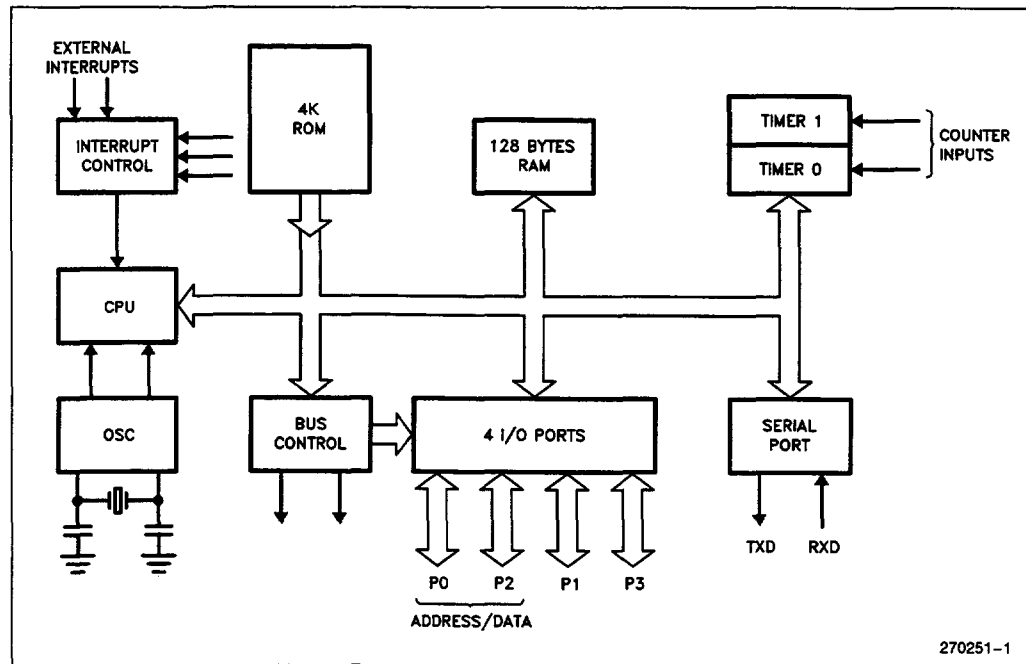
# *O microcontrolador 8051 o 8051*

- Os 8051 atuais
  - Vantagens de ensino
    - Micro de baixa complexidade
      - 44 mnemónicas *assembly*
        - Embora cada uma tenha declinações (diferentes opcodes)
        - 109 instruções
    - Implementa os conceitos relevantes
      - Classes de instruções
      - Modos de endereçamento
    - Periféricos
      - Baixa complexidade



# *O microcontrolador 8051* *arquitetura*

- Arquitetura do MCS 8051
  - O diagrama de blocos original



Fonte: <https://web.mit.edu/6.115/www/document/8051.pdf>

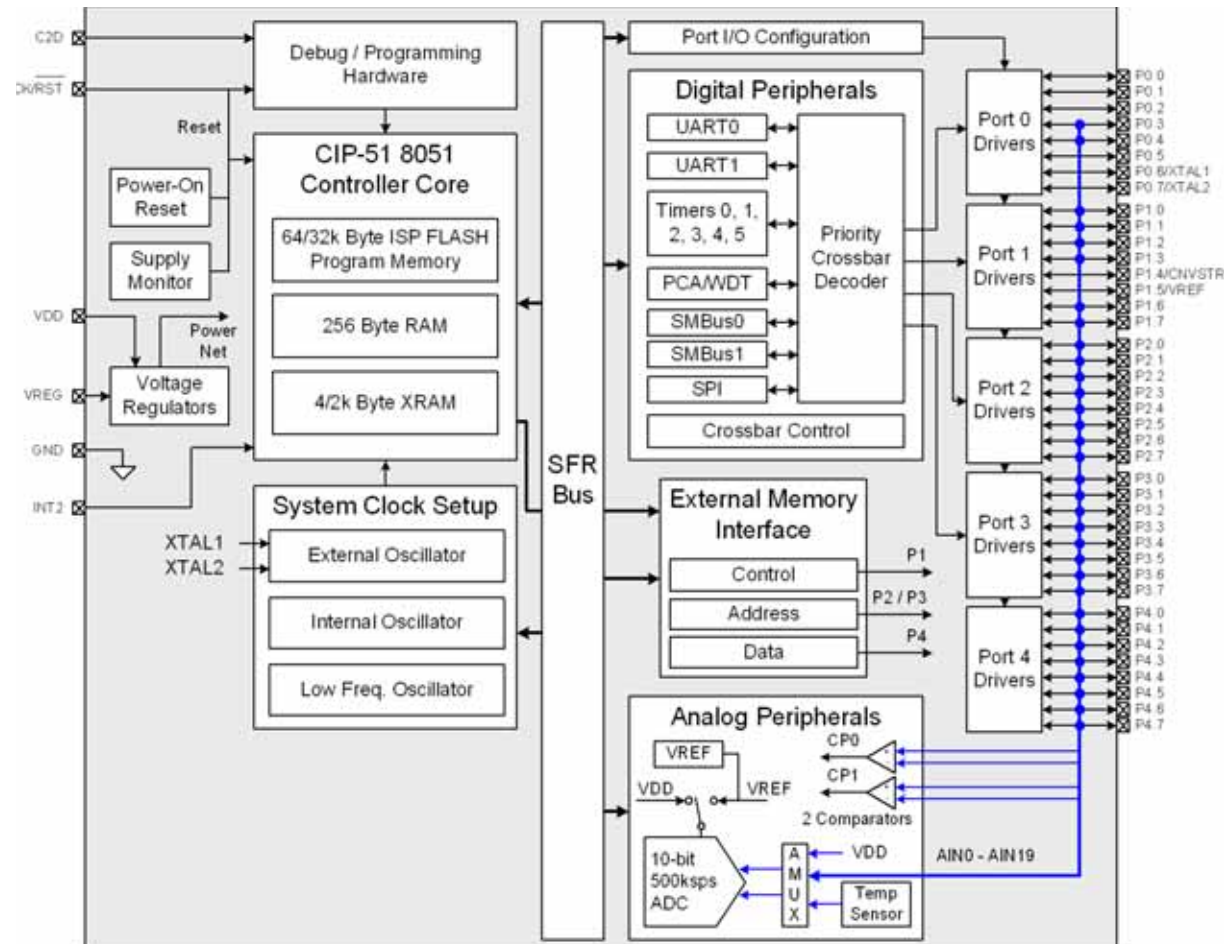


# *O microcontrolador 8051 arquitetura*



Embedded Systems  
Research Group

- Silicon Labs C8051F388
- Usado no nosso kit





# *O microcontrolador 8051*

## *arquitetura*

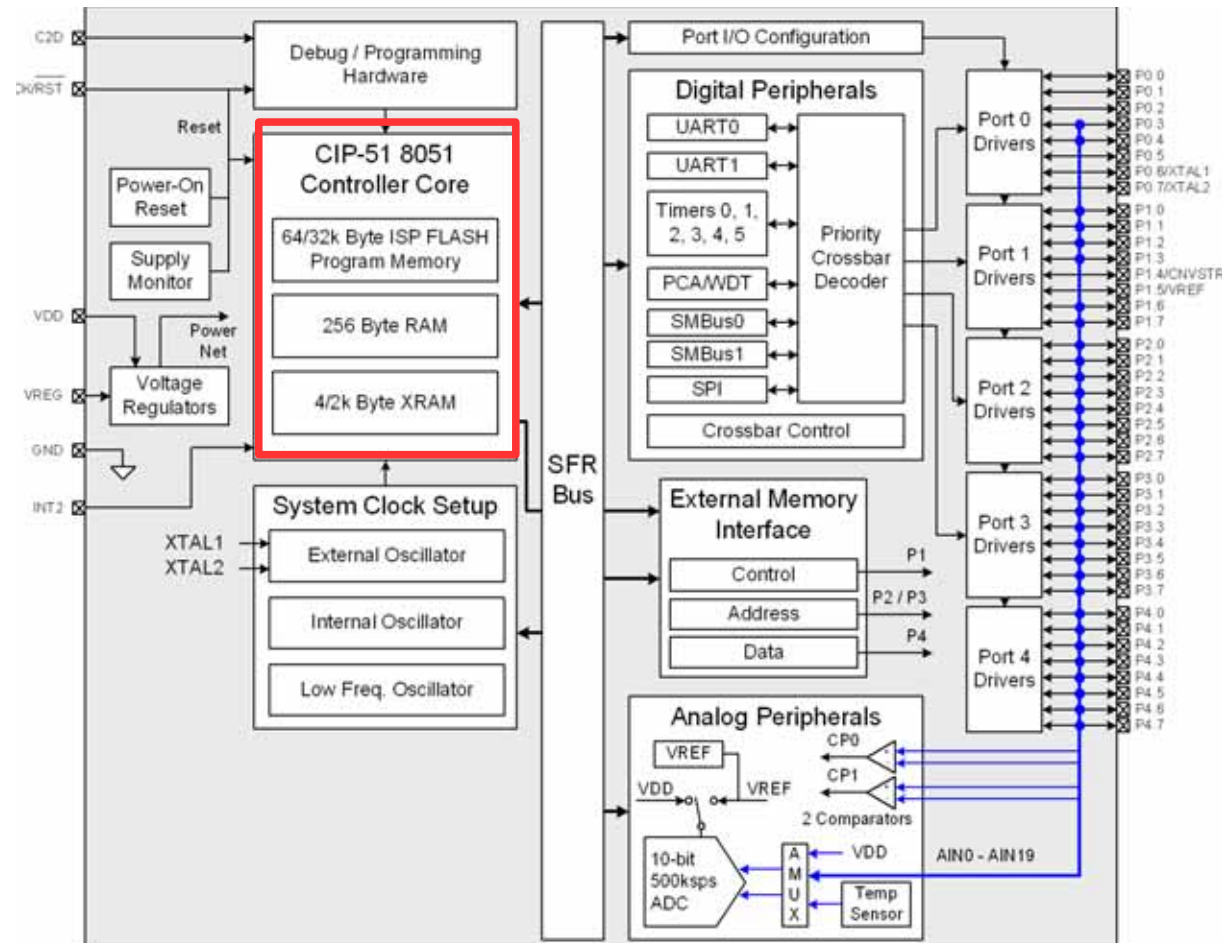
- Silicon Labs C8051F388
  - **Blocos**
    - *Core* do 8051
    - Relógio do sistema
    - Periféricos digitais
    - Memória externa
    - Periféricos analógicos



# *O microcontrolador 8051 o core do microcontrolador*



- *Core do microcontrolador*
- **Silicon Labs C8051F388**

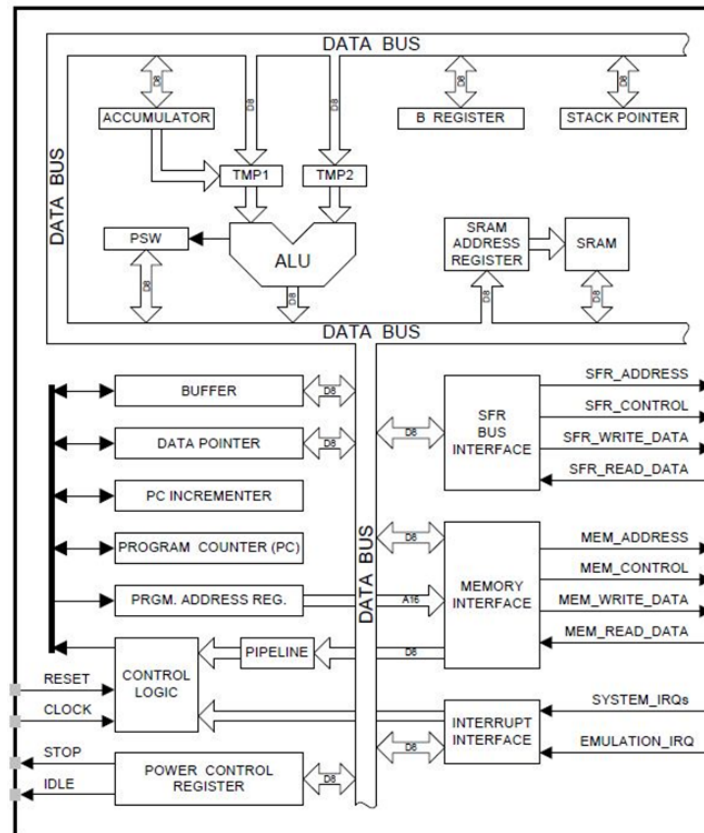




# *O microcontrolador 8051*

## *o core do microcontrolador*

- O *core* (CPU) do 8051
  - Versão Silicon Labs (designado CIP-51)





# *O microcontrolador 8051*

## *o core do microcontrolador*

- CIP51
  - Desempenho de 48 MIPS com relógio de 48 MHz
  - Arquitetura em *pipeline*
    - 70% das instruções executam em um ou dois ciclos de *clock* do sistema
    - Nenhuma instrução demora mais de oito ciclos de *clock*
  - Conjunto de instruções completamente compatível com o MSC-51
  - *Superset* dos periféricos do MSC-51
  - *On-chip debug hardware*

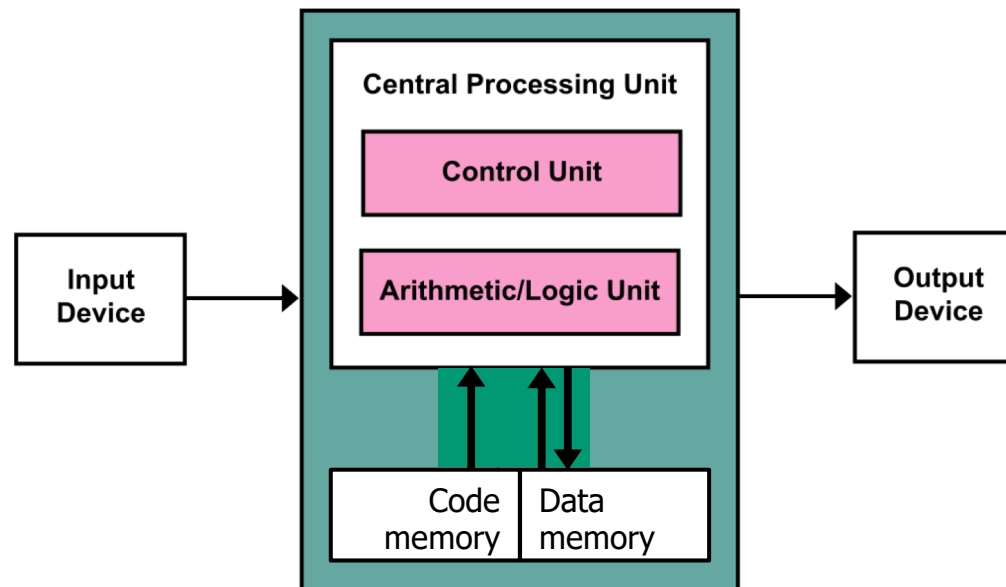




# *O microcontrolador 8051*

## *organização da memória*

- Organização da memória
  - 8051 usa variação do modelo von Neumann
    - Designado modelo Harvard
      - O código e os dados estão em memórias separadas





# *O microcontrolador 8051*

## *organização da memória*

- Organização da memória
  - 8051 e o modelo Harvard
    - Permite que o endereçamento seja separado
      - Logo, ter diferentes limites de memória
        - 64k para memória de código
          - Endereçamento de 2 bytes
            - O Program Counter (o que é?) é um registo de 2 bytes
        - 256 bytes para a memória interna de dados
          - Endereçamento de um byte
            - Ver instruções que lidam com memória de dados
    - Permite tb que as tecnologias de memória sejam diferentes
      - Flash (atualmente) para a memória de código
      - RAM para a memória de dados



# *O microcontrolador 8051*

## *organização da memória*

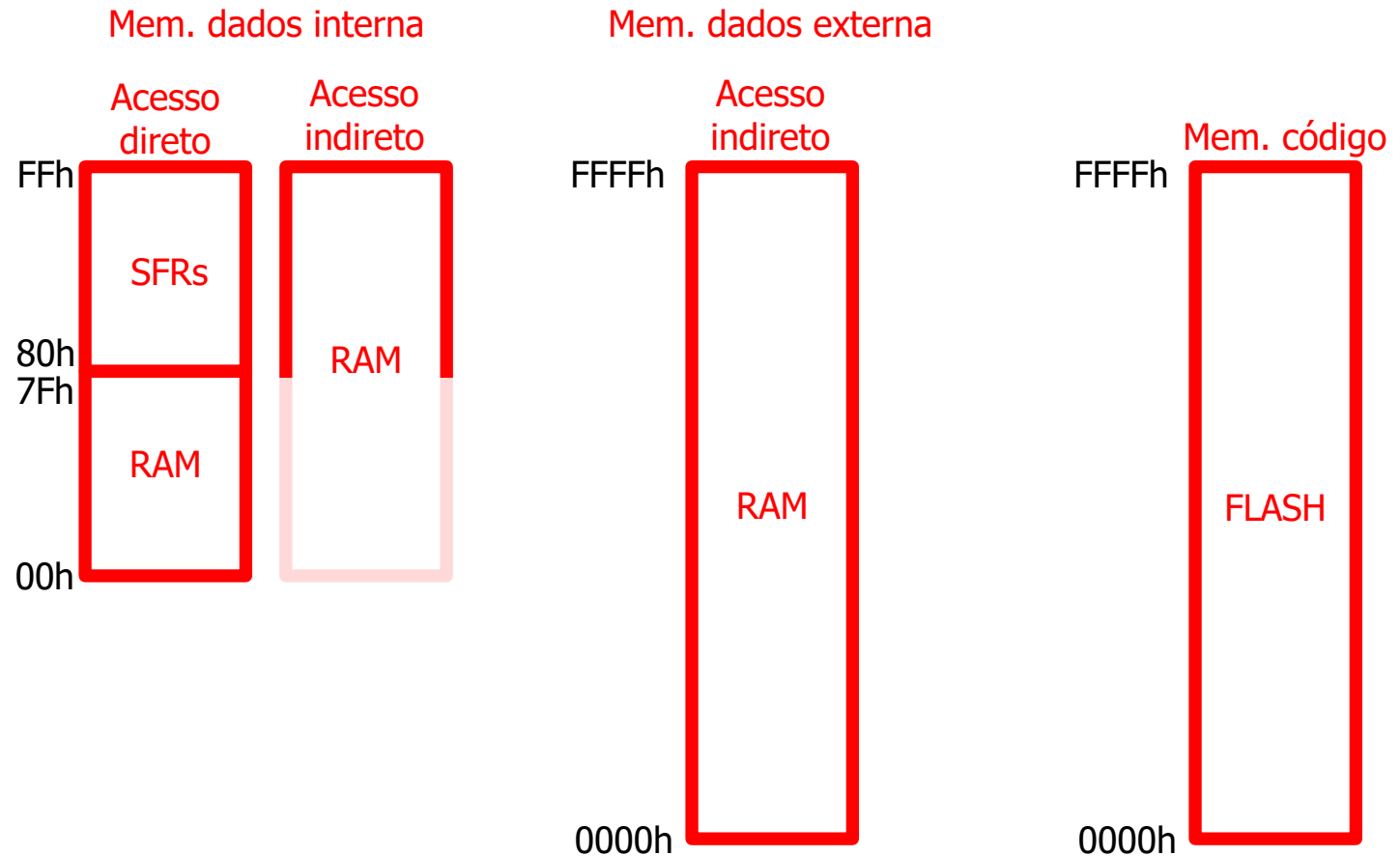
- Organização da memória
  - 8051 e o modelo Harvard
    - Não há incremento de desempenho
      - Ao contrário do que andam a ler fora das aulas (ver testes 2021/22)
- Outra arquiteturas Harvard
  - Ex. microcontrolador PIC
    - Barramentos de endereço e dados separados
      - Implica que se pode aceder às duas memórias simultaneamente
        - Aqui há ganho de desempenho



# *O microcontrolador 8051*

## *organização da memória*

- Organização da memória

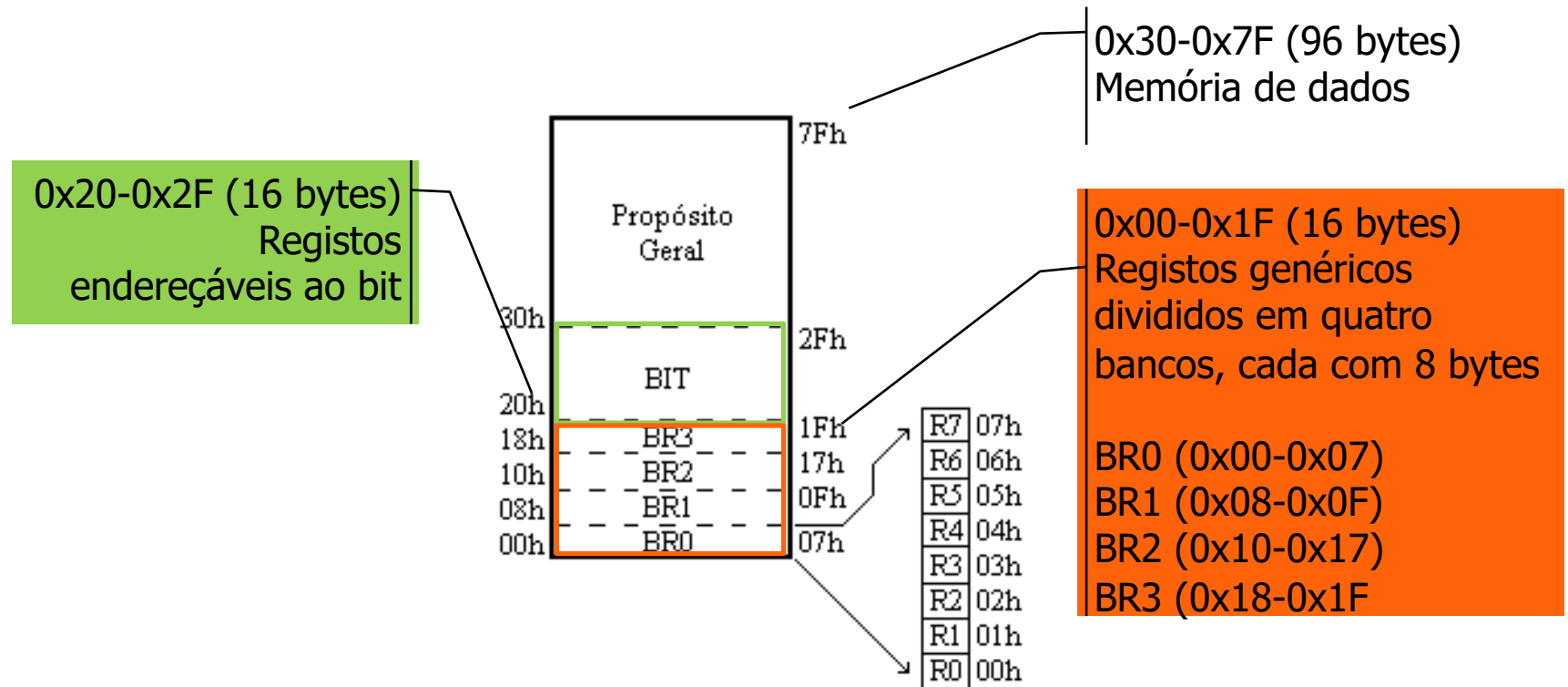




# *O microcontrolador 8051*

## *organização da memória*

- Organização da memória
  - Memória de dados interna
    - Os 128 bytes iniciais (0x00-0x7F)

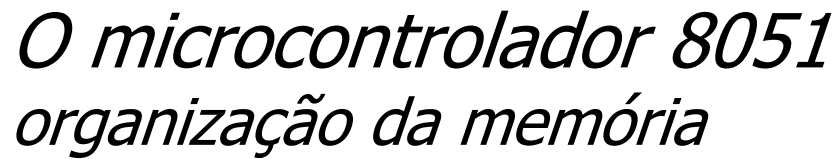




# *O microcontrolador 8051*

## *organização da memória*

- SFRs (Special Function Registers)
  - Registos (posições de memória) contendo
    - Configurações de periféricos
      - Definindo como o periférico funciona
        - Por exemplo: a resolução da amostragem de um ADC
    - Dados
      - Produzidos ou consumidos pelo periférico
        - Por exemplo: o dado convertido pelo ADC
  - Esta memória não é RAM disponível para variáveis



- SFRs (Special Function Registers)

- Visão simplificada (8051 original)

|                  | SFR MEMORY MAP                                      |      |        |        |     |     |  |      |                  |
|------------------|-----------------------------------------------------|------|--------|--------|-----|-----|--|------|------------------|
| F8               |                                                     |      |        |        |     |     |  |      | FF               |
| F0               | B                                                   |      |        |        |     |     |  |      | F7               |
| E8               |                                                     |      |        |        |     |     |  |      | EF               |
| E0               | ACC                                                 |      |        |        |     |     |  |      | E7               |
| D8               |                                                     |      |        |        |     |     |  |      | DF               |
| D0               | PSW                                                 |      |        |        |     |     |  |      | D7               |
| C8               | T2CON                                               |      | RCAP2L | RCAP2H | TL2 | TH2 |  |      | CF               |
| C0               |                                                     |      |        |        |     |     |  |      | C7               |
| B8               | IP                                                  |      |        |        |     |     |  |      | BF               |
| B0               | P3                                                  |      |        |        |     |     |  |      | B7               |
| A8               | IE                                                  |      |        |        |     |     |  |      | AF               |
| A0               | P2                                                  |      |        |        |     |     |  |      | A7               |
| 98               | SCON                                                | SBUF |        |        |     |     |  |      | 9F               |
| 90               | P1                                                  |      |        |        |     |     |  |      | 97               |
| 88               | TCON                                                | TMOD | TL0    | TL1    | TH0 | TH1 |  |      | 8F               |
| 80               | P0                                                  | SP   | DPL    | DPH    |     |     |  | PCON | 87               |
| MEMORY LOCATIONS | <div><div></div><div>8 BYTES</div><div></div></div> |      |        |        |     |     |  |      | MEMORY LOCATIONS |



# O microcontrolador 8051

## organização da memória

### • SFRs (Special Function Registers)

| Address | Page | 0(8)   | 1(9)     | 2(A)     | 3(B)     | 4(C)     | 5(D)     | 6(E)     | 7(F)    |
|---------|------|--------|----------|----------|----------|----------|----------|----------|---------|
| F8      |      | SPI0CN | PCA0L    | PCA0H    | PCA0CPL0 | PCA0CPH0 | PCA0CPL4 | PCA0CPH4 | VDM0CN  |
| F0      |      | B      | P0MDIN   | P1MDIN   | P2MDIN   | P3MDIN   | P4MDIN   | EIP1     | EIP2    |
| E8      |      | ADC0CN | PCA0CPL1 | PCA0CPH1 | PCA0CPL2 | PCA0CPH2 | PCA0CPL3 | PCA0CPH3 | RSTSRC  |
| E0      | 0    | ACC    | XBR0     | XBR1     | XBR2     | IT01CF   | SMOD1    | EIE1     | EIE2    |
|         | F    |        |          |          |          | CKCON1   |          |          |         |
| D8      |      | PCA0CN | PCA0MD   | PCA0CPM0 | PCA0CPM1 | PCA0CPM2 | PCA0CPM3 | PCA0CPM4 | P3SKIP  |
| D0      |      | PSW    | REF0CN   | SCON1    | SBUF1    | P0SKIP   | P1SKIP   | P2SKIP   |         |
| C8      | 0    | TMR2CN | REG01CN  | TMR2RLL  | TMR2RLH  | TMR2L    | TMR2H    | SMB0ADM  | SMB0ADR |
|         | F    | TMR5CN |          | TMR5RLL  | TMR5RLH  | TMR5L    | TMR5H    | SMB1ADM  | SMB1ADR |
| C0      | 0    | SMB0CN | SMB0CF   | SMB0DAT  | ADC0GTL  | ADC0GTH  | ADC0LTL  | ADC0LTH  | P4      |
|         | F    | SMB1CN | SMB1CF   | SMB1DAT  |          |          |          |          |         |
| B8      | 0    | IP     | CLKMUL   | AMX0N    | AMX0P    | ADC0CF   | ADC0L    | ADC0H    | SFRPAGE |
|         | F    |        | SMBTC    |          |          |          |          |          |         |
| B0      |      | P3     | OSCXC�   | OSCICN   | OSCICL   | SBRL11   | SBRLH1   | FLSCL    | FLKEY   |
| A8      |      | IE     | CLKSEL   | EMI0CN   |          | SBCON1   |          | P4MDOUT  | PFE0CN  |
| A0      |      | P2     | SPI0CFG  | SPI0CKR  | SPI0DAT  | P0MDOUT  | P1MDOUT  | P2MDOUT  | P3MDOUT |
| 98      |      | SCON0  | SBUF0    | CPT1CN   | CPT0CN   | CPT1MD   | CPT0MD   | CPT1MX   | CPT0MX  |
| 90      | 0    | P1     | TMR3CN   | TMR3RLL  | TMR3RLH  | TMR3L    | TMR3H    |          |         |
|         | F    |        | TMR4CN   | TMR4RLL  | TMR4RLH  | TMR4L    | TMR4H    |          |         |
| 88      |      | TCON   | TMOD     | TL0      | TL1      | TH0      | TH1      | CKCON    | PSCTL   |
| 80      |      | P0     | SP       | DPL      | DPH      | EMI0TC   | EMI0CF   | OSCLCN   | PCON    |
|         |      | 0(8)   | 1(9)     | 2(A)     | 3(B)     | 4(C)     | 5(D)     | 6(E)     | 7(F)    |

#### Notes:

1. SFR Addresses ending in 0x0 or 0x8 are bit-addressable locations and can be used with bitwise instructions.
2. Unless indicated otherwise, SFRs are available on both page 0 and page F.





# O microcontrolador 8051

## organização da memória

### • SFRs (Special Function Registers)

| Address | Page | 0(8)       | 1(9)      | 2(A)     | 3(B)     | 4(C)     | 5(D)     | 6(E)     | 7(F)    |
|---------|------|------------|-----------|----------|----------|----------|----------|----------|---------|
| F8      |      | SPI0CN     | PCA0L     | PCA0H    | PCA0CPL0 | PCA0CPH0 | PCA0CPL4 | PCA0CPH4 | VDM0CN  |
| F0      |      | <b>B</b>   | P0MDIN    | P1MDIN   | P2MDIN   | P3MDIN   | P4MDIN   | EIP1     | EIP2    |
| E8      |      | ADC0CN     | PCA0CPL1  | PCA0CPH1 | PCA0CPL2 | PCA0CPH2 | PCA0CPL3 | PCA0CPH3 | RSTSRC  |
| E0      | 0    | <b>ACC</b> | XBR0      | XBR1     | XBR2     | IT01CF   | SMOD1    | EIE1     | EIE2    |
|         | F    |            |           |          |          | CKCON1   |          |          |         |
| D8      |      | PCA0CN     | PCA0MD    | PCA0CPM0 | PCA0CPM1 | PCA0CPM2 | PCA0CPM3 | PCA0CPM4 | P3SKIP  |
| D0      |      | <b>PSW</b> | REF0CN    | SCON1    | SBUF1    | P0SKIP   | P1SKIP   | P2SKIP   |         |
| C8      | 0    | TMR2CN     | REG01CN   | TMR2RLL  | TMR2RLH  | TMR2L    | TMR2H    | SMB0ADM  | SMB0ADR |
|         | F    | TMR5CN     |           | TMR5RLL  | TMR5RLH  | TMR5L    | TMR5H    | SMB1ADM  | SMB1ADR |
| C0      | 0    | SMB0CN     | SMB0CF    | SMB0DAT  | ADC0GTL  | ADC0GTH  | ADC0LTL  | ADC0LTH  | P4      |
|         | F    | SMB1CN     | SMB1CF    | SMB1DAT  |          |          |          |          |         |
| B8      | 0    | <b>IP</b>  | CLKMUL    | AMX0N    | AMX0P    | ADC0CF   | ADC0L    | ADC0H    | SFRPAGE |
|         | F    |            | SMBTC     |          |          |          |          |          |         |
| B0      |      | P3         | OSCXC�    | OSCICN   | OSCICL   | SBRL11   | SBRLH1   | FLSCL    | FLKEY   |
| A8      |      | <b>IE</b>  | CLKSEL    | EMI0CN   |          | SBCON1   |          | P4MDOUT  | PFE0CN  |
| A0      |      | P2         | SPI0CFG   | SPI0CKR  | SPI0DAT  | P0MDOUT  | P1MDOUT  | P2MDOUT  | P3MDOUT |
| 98      |      | SCON0      | SBUF0     | CPT1CN   | CPT0CN   | CPT1MD   | CPT0MD   | CPT1MX   | CPT0MX  |
| 90      | 0    | P1         | TMR3CN    | TMR3RLL  | TMR3RLH  | TMR3L    | TMR3H    |          |         |
|         | F    |            | TMR4CN    | TMR4RLL  | TMR4RLH  | TMR4L    | TMR4H    |          |         |
| 88      |      | TCON       | TMOD      | TL0      | TL1      | TH0      | TH1      | CKCON    | PSCTL   |
| 80      |      | <b>P0</b>  | <b>SP</b> | DPL      | DPH      | EMI0TC   | EMI0CF   | OSCLCN   | PCON    |
|         |      | 0(8)       | 1(9)      | 2(A)     | 3(B)     | 4(C)     | 5(D)     | 6(E)     | 7(F)    |

#### Notes:

1. SFR Addresses ending in 0x0 or 0x8 are bit-addressable locations and can be used with bitwise instructions.
2. Unless indicated otherwise, SFRs are available on both page 0 and page F.



# *O microcontrolador 8051*

## *organização da memória*

- SFRs (Special Function Registers)
  - Páginas de SFRs
    - A arquitetura do CIP-51 implementa o conceito de páginas, como na figura abaixo
      - O nosso micro só implementa duas páginas (0 e F)

| 0       |         |          |          |          |          |          |          |          |         |
|---------|---------|----------|----------|----------|----------|----------|----------|----------|---------|
| C       |         |          |          |          |          |          |          |          |         |
| 0       |         |          |          |          |          |          |          |          |         |
| SP      | SP      | 0(8)     | 1(9)     | 2(A)     | 3(B)     | 4(C)     | 5(D)     | 6(E)     | 7(F)    |
| ADCON   | ADCON   | SPIOCN   | PCA0L    | PCA0H    | PCA0CPL0 | PCA0CPH0 | PCA0CPL4 | PCA0CPH4 | VDMOCN  |
| PCADCON | PCADCON | B        | P0MDIN   | P1MDIN   | P2MDIN   | P3MDIN   | P4MDIN   | EIP1     | EIP2    |
| PCADCON | PCADCON | PCA0CPL1 | PCA0CPH1 | PCA0CPL2 | PCA0CPH2 | PCA0CPL3 | PCA0CPH3 | RSTSRC   |         |
| TMR0CN  | TMR0CN  | ACC      | XBR0     | XBR1     | XBR2     | IT01CF   | SMOD1    | EIE1     | EIE2    |
| TMR1CN  | TMR1CN  | PCA0CN   | PCA0MD   | PCA0CPM0 | PCA0CPM1 | CKCON1   |          |          |         |
| TMR2CN  | TMR2CN  | PSW      | REF0CN   | SCON1    | SBUF1    | P0SKIP   | P1SKIP   | P2SKIP   | P3SKIP  |
| TMR3CN  | TMR3CN  | TMR2CN   | REG01CN  | TMR2RLL  | TMR2RLH  | TMR2L    | TMR2H    | SMB0ADM  | SMB0ADR |
| TMR4CN  | TMR4CN  | TMR5CN   |          | TMR5RLL  | TMR5RLH  | TMR5L    | TMR5H    | SMB1ADM  | SMB1ADR |
| TMR5CN  | TMR5CN  | SMB0CN   | SMB0CF   | SMB0DAT  | ADC0GTL  | ADC0GTH  | ADC0LTL  | ADC0LTH  | P4      |
| SCON0   | SCON0   | SMB1CN   | SMB1CF   | SMB1DAT  | AMX0N    | AMX0P    | ADC0CF   | ADC0L    | ADC0H   |
| SCON1   | SCON1   | IP       | CLKMUL   |          |          |          |          |          | SFRPAGE |
| SCON2   | SCON2   |          | SMBTC    |          |          |          |          |          |         |
| TCN0    | TCN0    | P3       | OSCXCN   | OSCICN   | OSCICL   | SBRL11   | SBRLH1   | FLSCL    | FLKEY   |
| TCN1    | TCN1    | IE       | CLKSEL   | EMI0CN   |          | SBCON1   |          | P4MDOUT  | PFE0CN  |
| TCN2    | TCN2    | P2       | SPI0CFG  | SPI0CKR  | SPI0DAT  | P0MDOUT  | P1MDOUT  | P2MDOUT  | P3MDOUT |
| TCN3    | TCN3    | SCON0    | SBUF0    | CPT1CN   | CPT0CN   | CPT1MD   | CPT0MD   | CPT1MX   | CPT0MX  |
| TCN4    | TCN4    | P1       | TMR3CN   | TMR3RLL  | TMR3RLH  | TMR3L    | TMR3H    |          |         |
| TCN5    | TCN5    |          | TMR4CN   | TMR4RLL  | TMR4RLH  | TMR4L    | TMR4H    |          |         |
| TCN6    | TCN6    | TCON     | TMOD     | TL0      | TL1      | TH0      | TH1      | CKCON    | PSCTL   |
| TCN7    | TCN7    | P0       | SP       | DPL      | DPH      | EMI0TC   | EMI0CF   | OSCLCN   | PCON    |
| TCN8    | TCN8    | 0(8)     | 1(9)     | 2(A)     | 3(B)     | 4(C)     | 5(D)     | 6(E)     | 7(F)    |



# O microcontrolador 8051

## organização da memória

- SFRs (Special Function Registers)
- Páginas de registos

| Address | Page   | 0(8)   | 1(9)            | 2(A)     | 3(B)     | 4(C)             | 5(D)     | 6(E)     | 7(F)    |
|---------|--------|--------|-----------------|----------|----------|------------------|----------|----------|---------|
| F8      |        | SPI0CN | PCA0L           | PCA0H    | PCA0CPL0 | PCA0CPH0         | PCA0CPL4 | PCA0CPH4 | VDM0CN  |
| F0      |        | B      | P0MDIN          | P1MDIN   | P2MDIN   | P3MDIN           | P4MDIN   | EIP1     | EIP2    |
| E8      |        | ADC0CN | PCA0CPL1        | PCA0CPH1 | PCA0CPL2 | PCA0CPH2         | PCA0CPL3 | PCA0CPH3 | RSTSRC  |
| E0      | 0<br>F | ACC    | XBR0            | XBR1     | XBR2     | IT01CF<br>CKCON1 | SMOD1    | EIE1     | EIE2    |
| D8      |        | PCA0CN | PCA0MD          | PCA0CPM0 | PCA0CPM1 | PCA0CPM2         | PCA0CPM3 | PCA0CPM4 | P3SKIP  |
| D0      |        | PSW    | REF0CN          | SCON1    | SBUF1    | P0SKIP           | P1SKIP   | P2SKIP   |         |
| C8      | 0<br>F | TMR2CN | REG01CN         | TMR2RLL  | TMR2RLH  | TMR2L            | TMR2H    | SMB0ADM  | SMB0ADR |
|         |        | TMR5CN |                 | TMR5RLL  | TMR5RLH  | TMR5L            | TMR5H    | SMB1ADM  | SMB1ADR |
| C0      | 0<br>F | SMB0CN | SMB0CF          | SMB0DAT  | ADC0GTL  | ADC0GTH          | ADC0LTL  | ADC0LTH  | P4      |
|         |        | SMB1CN | SMB1CF          | SMB1DAT  |          |                  |          |          |         |
| B8      | 0<br>F | IP     | CLKMUL<br>SMBTC | AMX0N    | AMX0P    | ADC0CF           | ADC0L    | ADC0H    | SFRPAGE |
| B0      |        | P3     | OSCXCN          | OSCICN   | OSCICL   | SBRL1            |          |          |         |
| A8      |        | IE     | CLKSEL          | EMI0CN   |          | SBCON1           |          | P4MDOUT  | PFE0CN  |
| A0      |        | P2     | SPI0CFG         | SPI0CKR  | SPI0DAT  | P0MDOUT          | P1MDOUT  | P2MDOUT  | P3MDOUT |
| 98      |        | SCON0  | SBUF0           | CPT1CN   | CPT0CN   | CPT1MD           | CPT0MD   | CPT1MX   | CPT0MX  |
| 90      | 0<br>F | P1     | TMR3CN          | TMR3RLL  | TMR3RLH  | TMR3L            | TMR3H    |          |         |
|         |        |        | TMR4CN          | TMR4RLL  | TMR4RLH  | TMR4L            | TMR4H    |          |         |
| 88      |        | TCON   | TMOD            | TL0      | TL1      | TH0              | TH1      | CKCON    | PSCTL   |
| 80      |        | P0     | SP              | DPL      | DPH      | EMI0TC           | EMI0CF   | OSCLCN   | PCON    |
|         |        | 0(8)   | 1(9)            | 2(A)     | 3(B)     | 4(C)             | 5(D)     | 6(E)     | 7(F)    |



# *O microcontrolador 8051*

## *organização da memória*

- Memória de código
  - Recebe o código do programa carregado através do *debugger de hardware*
  - Acessível durante a execução do programa através da instrução *assembly* **movc**
    - Podendo ser usada para dados não voláteis (constantes)



# *O microcontrolador 8051*

## *organização da memória*

- Memória de código
  - Contém o código do programa
    - Representação binária das instruções *assembly*
      - Formato genérico

Exemplo

|  | Instrução | Destino | Origem     |
|--|-----------|---------|------------|
|  | MOV       | R0,     | A ; R0 ← A |

- Convertida em binário seria

11111000 ; R0 ← A

- Curiosamente esta instrução só ocupa 1 byte na memória

- Outro exemplo

|          |          |          |
|----------|----------|----------|
| MOV      | P2,      | #0FFh    |
| 01110101 | 10100000 | 11111111 |

- Esta instrução ocupa 3 bytes na memória

- É uma instrução que o CPU vai fazer o *fetch*, *decode* e *execute*



# O microcontrolador 8051 o conjunto de instruções

- Exemplo
  - Todas as formas da instrução **mov**

| Instructions      | OpCode | Bytes |
|-------------------|--------|-------|
| MOV @R0,# data    | 0x76   | 2     |
| MOV @R1,# data    | 0x77   | 2     |
| MOV @R0,A         | 0xF6   | 1     |
| MOV @R1,A         | 0xF7   | 1     |
| MOV @R0,iram addr | 0xA6   | 2     |
| MOV @R1,iram addr | 0xA7   | 2     |
| MOV A,# data      | 0x74   | 2     |
| MOV A,@R0         | 0xE6   | 1     |
| MOV A,@R1         | 0xE7   | 1     |
| MOV A,R0          | 0xE8   | 1     |
| MOV A,R1          | 0xE9   | 1     |
| MOV A,R2          | 0xEA   | 1     |
| MOV A,R3          | 0xEB   | 1     |
| MOV A,R4          | 0xEC   | 1     |
| MOV A,R5          | 0xED   | 1     |
| MOV A,R6          | 0xEE   | 1     |
| MOV A,R7          | 0xEF   | 1     |
| MOV A,iram addr   | 0xE5   | 2     |
| MOV C,bit addr    | 0xA2   | 2     |
| MOV DPTR,# data16 | 0x90   | 3     |

| Instructions  | OpCode | Bytes |
|---------------|--------|-------|
| MOV R0,# data | 0x78   | 2     |
| MOV R1,# data | 0x79   | 2     |
| MOV R2,# data | 0x7A   | 2     |
| MOV R3,# data | 0x7B   | 2     |
| MOV R4,# data | 0x7C   | 2     |
| MOV R5,# data | 0x7D   | 2     |
| MOV R6,# data | 0x7E   | 2     |
| MOV R7,# data | 0x7F   | 2     |
| MOV R0,A      | 0xF8   | 1     |
| MOV R1,A      | 0xF9   | 1     |
| MOV R2,A      | 0xFA   | 1     |
| MOV R3,A      | 0xFB   | 1     |
| MOV R4,A      | 0xFC   | 1     |
| MOV R5,A      | 0xFD   | 1     |
| MOV R6,A      | 0xFE   | 1     |
| MOV R7,A      | 0xFF   | 1     |

| Instructions            | OpCode | Bytes |
|-------------------------|--------|-------|
| MOV R0,iram addr        | 0xA8   | 2     |
| MOV R1,iram addr        | 0xA9   | 2     |
| MOV R2,iram addr        | 0xAA   | 2     |
| MOV R3,iram addr        | 0xAB   | 2     |
| MOV R4,iram addr        | 0xAC   | 2     |
| MOV R5,iram addr        | 0xAD   | 2     |
| MOV R6,iram addr        | 0xAE   | 2     |
| MOV R7,iram addr        | 0xAF   | 2     |
| MOV bit addr,C          | 0x92   | 2     |
| MOV iram addr,# data    | 0x75   | 3     |
| MOV iram addr,@R0       | 0x86   | 2     |
| MOV iram addr,@R1       | 0x87   | 2     |
| MOV iram addr,R0        | 0x88   | 2     |
| MOV iram addr,R1        | 0x89   | 2     |
| MOV iram addr,R2        | 0x8A   | 2     |
| MOV iram addr,R3        | 0x8B   | 2     |
| MOV iram addr,R4        | 0x8C   | 2     |
| MOV iram addr,R5        | 0x8D   | 2     |
| MOV iram addr,R6        | 0x8E   | 2     |
| MOV iram addr,R7        | 0x8F   | 2     |
| MOV iram addr,A         | 0xF5   | 2     |
| MOV iram addr,iram addr | 0x85   | 3     |

- Bom manual do *instruction set*

[https://www.keil.com/support/man/docs/is51/is51\\_opcodes.htm](https://www.keil.com/support/man/docs/is51/is51_opcodes.htm)



# *O microcontrolador 8051*

## *o conjunto de instruções*

- O conjunto de instruções
  - Tipos de instruções
    - Operações aritméticas
    - Operações lógicas
    - Transferência de dados
    - Manipulação booleana
    - Controlo de fluxo



# *O microcontrolador 8051*

## *o conjunto de instruções*

- O conjunto de instruções
  - Tipos de endereçamento
    - Registo
    - Direto
    - Indireto
    - Imediato com constante
    - Relativo
    - Absoluto
    - Longo
    - Indexado

| Addressing Modes   | Instruction          |
|--------------------|----------------------|
| Register           | MOV A, B             |
| Direct             | MOV 30H,A            |
| Indirect           | ADD A,@R0            |
| Immediate Constant | ADD A,#80H           |
| Relative*          | SJMP +127/-128 of PC |
| Absolute*          | AJMP within 2K       |
| Long*              | LJMP FAR             |
| Indexed            | MOVC A,@A+PC         |

\* Relativo a instruções de controlo de fluxo





# *O microcontrolador 8051*

## *o conjunto de instruções*

- Operações aritméticas

| Mnemonic                     | Description                              | Bytes | Clock Cycles |
|------------------------------|------------------------------------------|-------|--------------|
| <b>Arithmetic Operations</b> |                                          |       |              |
| ADD A, Rn                    | Add register to A                        | 1     | 1            |
| ADD A, direct                | Add direct byte to A                     | 2     | 2            |
| ADD A, @Ri                   | Add indirect RAM to A                    | 1     | 2            |
| ADD A, #data                 | Add immediate to A                       | 2     | 2            |
| ADDC A, Rn                   | Add register to A with carry             | 1     | 1            |
| ADDC A, direct               | Add direct byte to A with carry          | 2     | 2            |
| ADDC A, @Ri                  | Add indirect RAM to A with carry         | 1     | 2            |
| ADDC A, #data                | Add immediate to A with carry            | 2     | 2            |
| SUBB A, Rn                   | Subtract register from A with borrow     | 1     | 1            |
| SUBB A, direct               | Subtract direct byte from A with borrow  | 2     | 2            |
| SUBB A, @Ri                  | Subtract indirect RAM from A with borrow | 1     | 2            |
| SUBB A, #data                | Subtract immediate from A with borrow    | 2     | 2            |
| INC A                        | Increment A                              | 1     | 1            |
| INC Rn                       | Increment register                       | 1     | 1            |
| INC direct                   | Increment direct byte                    | 2     | 2            |
| INC @Ri                      | Increment indirect RAM                   | 1     | 2            |
| DEC A                        | Decrement A                              | 1     | 1            |
| DEC Rn                       | Decrement register                       | 1     | 1            |
| DEC direct                   | Decrement direct byte                    | 2     | 2            |
| DEC @Ri                      | Decrement indirect RAM                   | 1     | 2            |
| INC DPTR                     | Increment Data Pointer                   | 1     | 1            |
| MUL AB                       | Multiply A and B                         | 1     | 4            |
| DIV AB                       | Divide A by B                            | 1     | 8            |
| DA A                         | Decimal adjust A                         | 1     | 1            |



# *O microcontrolador 8051*

## *o conjunto de instruções*

- Operações lógicas

| Mnemonic                  | Description                           | Bytes | Clock Cycles |
|---------------------------|---------------------------------------|-------|--------------|
| <b>Logical Operations</b> |                                       |       |              |
| ANL A, Rn                 | AND Register to A                     | 1     | 1            |
| ANL A, direct             | AND direct byte to A                  | 2     | 2            |
| ANL A, @Ri                | AND indirect RAM to A                 | 1     | 2            |
| ANL A, #data              | AND immediate to A                    | 2     | 2            |
| ANL direct, A             | AND A to direct byte                  | 2     | 2            |
| ANL direct, #data         | AND immediate to direct byte          | 3     | 3            |
| ORL A, Rn                 | OR Register to A                      | 1     | 1            |
| ORL A, direct             | OR direct byte to A                   | 2     | 2            |
| ORL A, @Ri                | OR indirect RAM to A                  | 1     | 2            |
| ORL A, #data              | OR immediate to A                     | 2     | 2            |
| ORL direct, A             | OR A to direct byte                   | 2     | 2            |
| ORL direct, #data         | OR immediate to direct byte           | 3     | 3            |
| XRL A, Rn                 | Exclusive-OR Register to A            | 1     | 1            |
| XRL A, direct             | Exclusive-OR direct byte to A         | 2     | 2            |
| XRL A, @Ri                | Exclusive-OR indirect RAM to A        | 1     | 2            |
| XRL A, #data              | Exclusive-OR immediate to A           | 2     | 2            |
| XRL direct, A             | Exclusive-OR A to direct byte         | 2     | 2            |
| XRL direct, #data         | Exclusive-OR immediate to direct byte | 3     | 3            |
| CLR A                     | Clear A                               | 1     | 1            |
| CPL A                     | Complement A                          | 1     | 1            |
| RL A                      | Rotate A left                         | 1     | 1            |
| RLC A                     | Rotate A left through Carry           | 1     | 1            |
| RR A                      | Rotate A right                        | 1     | 1            |
| RRC A                     | Rotate A right through Carry          | 1     | 1            |
| SWAP A                    | Swap nibbles of A                     | 1     | 1            |



# *O microcontrolador 8051*

## *o conjunto de instruções*

- Transferência de dados

| Mnemonic             | Description                                | Bytes | Clock Cycles |
|----------------------|--------------------------------------------|-------|--------------|
| <b>Data Transfer</b> |                                            |       |              |
| MOV A, Rn            | Move Register to A                         | 1     | 1            |
| MOV A, direct        | Move direct byte to A                      | 2     | 2            |
| MOV A, @Ri           | Move indirect RAM to A                     | 1     | 2            |
| MOV A, #data         | Move immediate to A                        | 2     | 2            |
| MOV Rn, A            | Move A to Register                         | 1     | 1            |
| MOV Rn, direct       | Move direct byte to Register               | 2     | 2            |
| MOV Rn, #data        | Move immediate to Register                 | 2     | 2            |
| MOV direct, A        | Move A to direct byte                      | 2     | 2            |
| MOV direct, Rn       | Move Register to direct byte               | 2     | 2            |
| MOV direct, direct   | Move direct byte to direct byte            | 3     | 3            |
| MOV direct, @Ri      | Move indirect RAM to direct byte           | 2     | 2            |
| MOV direct, #data    | Move immediate to direct byte              | 3     | 3            |
| MOV @Ri, A           | Move A to indirect RAM                     | 1     | 2            |
| MOV @Ri, direct      | Move direct byte to indirect RAM           | 2     | 2            |
| MOV @Ri, #data       | Move immediate to indirect RAM             | 2     | 2            |
| MOV DPTR, #data16    | Load DPTR with 16-bit constant             | 3     | 3            |
| MOVC A, @A+DPTR      | Move code byte relative DPTR to A          | 1     | 3            |
| MOVC A, @A+PC        | Move code byte relative PC to A            | 1     | 3            |
| MOVX A, @Ri          | Move external data (8-bit address) to A    | 1     | 3            |
| MOVX @Ri, A          | Move A to external data (8-bit address)    | 1     | 3            |
| MOVX A, @DPTR        | Move external data (16-bit address) to A   | 1     | 3            |
| MOVX @DPTR, A        | Move A to external data (16-bit address)   | 1     | 3            |
| PUSH direct          | Push direct byte onto stack                | 2     | 2            |
| POP direct           | Pop direct byte from stack                 | 2     | 2            |
| XCH A, Rn            | Exchange Register with A                   | 1     | 1            |
| XCH A, direct        | Exchange direct byte with A                | 2     | 2            |
| XCH A, @Ri           | Exchange indirect RAM with A               | 1     | 2            |
| XCHD A, @Ri          | Exchange low nibble of indirect RAM with A | 1     | 2            |



# *O microcontrolador 8051*

## *o conjunto de instruções*

- Manipulação booleana

| Mnemonic                    | Description                           | Bytes | Clock Cycles |
|-----------------------------|---------------------------------------|-------|--------------|
| <b>Boolean Manipulation</b> |                                       |       |              |
| CLR C                       | Clear Carry                           | 1     | 1            |
| CLR bit                     | Clear direct bit                      | 2     | 2            |
| SETB C                      | Set Carry                             | 1     | 1            |
| SETB bit                    | Set direct bit                        | 2     | 2            |
| CPL C                       | Complement Carry                      | 1     | 1            |
| CPL bit                     | Complement direct bit                 | 2     | 2            |
| ANL C, bit                  | AND direct bit to Carry               | 2     | 2            |
| ANL C, /bit                 | AND complement of direct bit to Carry | 2     | 2            |
| ORL C, bit                  | OR direct bit to carry                | 2     | 2            |
| ORL C, /bit                 | OR complement of direct bit to Carry  | 2     | 2            |
| MOV C, bit                  | Move direct bit to Carry              | 2     | 2            |
| MOV bit, C                  | Move Carry to direct bit              | 2     | 2            |



# *O microcontrolador 8051*

## *o conjunto de instruções*

- Controlo de fluxo

| Mnemonic                                                                                             | Description                                         | Bytes | Clock Cycles |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------|-------|--------------|
| <b>Program Flow</b>                                                                                  |                                                     |       |              |
| Timings are listed with the PFE on and FLRT = 0. Extra cycles are required for branches if FLRT = 1. |                                                     |       |              |
| JC rel                                                                                               | Jump if Carry is set                                | 2     | 2/4          |
| JNC rel                                                                                              | Jump if Carry is not set                            | 2     | 2/4          |
| JB bit, rel                                                                                          | Jump if direct bit is set                           | 3     | 3/5          |
| JNB bit, rel                                                                                         | Jump if direct bit is not set                       | 3     | 3/5          |
| JBC bit, rel                                                                                         | Jump if direct bit is set and clear bit             | 3     | 3/5          |
| ACALL addr11                                                                                         | Absolute subroutine call                            | 2     | 4            |
| LCALL addr16                                                                                         | Long subroutine call                                | 3     | 5            |
| RET                                                                                                  | Return from subroutine                              | 1     | 6            |
| RETI                                                                                                 | Return from interrupt                               | 1     | 6            |
| AJMP addr11                                                                                          | Absolute jump                                       | 2     | 4            |
| LJMP addr16                                                                                          | Long jump                                           | 3     | 5            |
| SJMP rel                                                                                             | Short jump (relative address)                       | 2     | 4            |
| JMP @A+DPTR                                                                                          | Jump indirect relative to DPTR                      | 1     | 4            |
| JZ rel                                                                                               | Jump if A equals zero                               | 2     | 2/4          |
| JNZ rel                                                                                              | Jump if A does not equal zero                       | 2     | 2/4          |
| CJNE A, direct, rel                                                                                  | Compare direct byte to A and jump if not equal      | 3     | 4/6          |
| CJNE A, #data, rel                                                                                   | Compare immediate to A and jump if not equal        | 3     | 3/5          |
| CJNE Rn, #data, rel                                                                                  | Compare immediate to Register and jump if not equal | 3     | 3/5          |
| CJNE @Ri, #data, rel                                                                                 | Compare immediate to indirect and jump if not equal | 3     | 4/6          |
| DJNZ Rn, rel                                                                                         | Decrement Register and jump if not zero             | 2     | 2/4          |
| DJNZ direct, rel                                                                                     | Decrement direct byte and jump if not zero          | 3     | 3/5          |
| NOP                                                                                                  | No operation                                        | 1     | 1            |



# *O microcontrolador 8051*

## *programação em C*

- O código C que vamos criar
  - É traduzido em *assembly*
    - Os programas são formados por instruções do conjunto apresentado nos *slides* anteriores
  - Tendo em conta a organização da memória do uC
    - Podem ser declaradas variáveis nas diferentes memórias



# *O microcontrolador 8051*

## *programação em C*

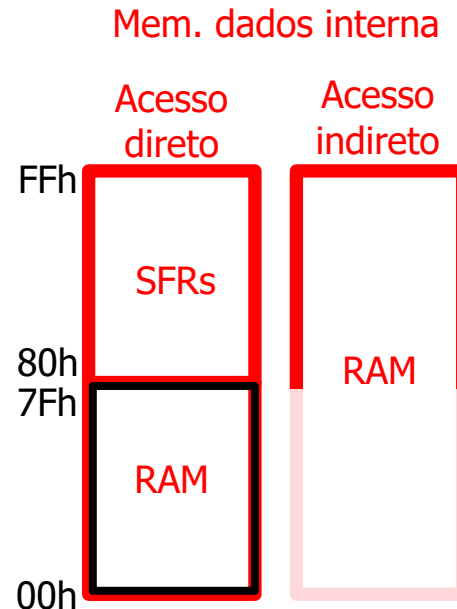
- Declaração de variáveis
  - Modelo de memória
    - Tem a ver com o espaço de memória disponível
      - Tem implicações ao nível do endereçamento e do tipo de memória usada
        - Visível em assembly...
        - Em C temos de ler o que diz o manual do compilador...
  - Modelos de memória
    - Small
      - Usa memória interna (128 bytes)
    - Medium
      - Usa memória externa ( um bloco de 256 bytes)
    - Large
      - Usa memória externa ( até 64 kbytes)



# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis
  - Mem. interna
    - Espaço de memória genérico (`__data`, `__near`)
      - Usado por defeito no modelo de memória **small**
        - `ex`
          - `__data unsigned char test_data;`



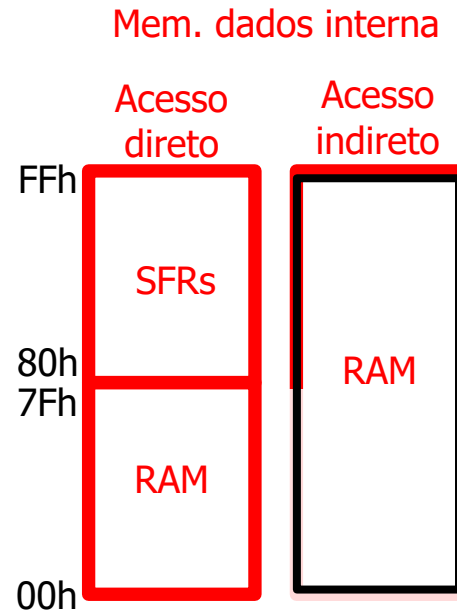




# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis
  - Mem. interna
    - Espaço endereçável indiretamente (`__idata`)
      - Pode ser nos 128 bytes mais baixos ou nos 128 mais altos
        - ex
          - `__idata unsigned char test_idata;`

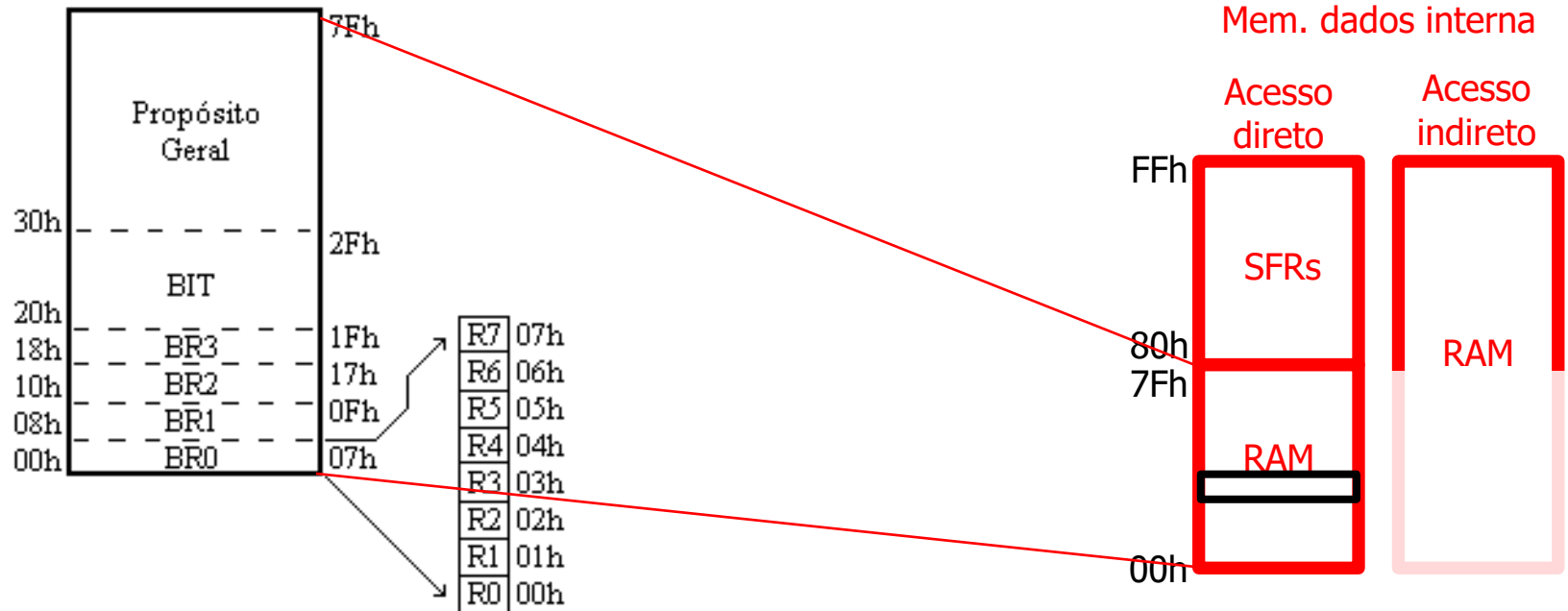




# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis
  - Mem. interna
    - Espaço de memória endereçável ao bit (`__bit`)
      - Variáveis do tipo bit
        - ex
          - `__bit test_bit;`

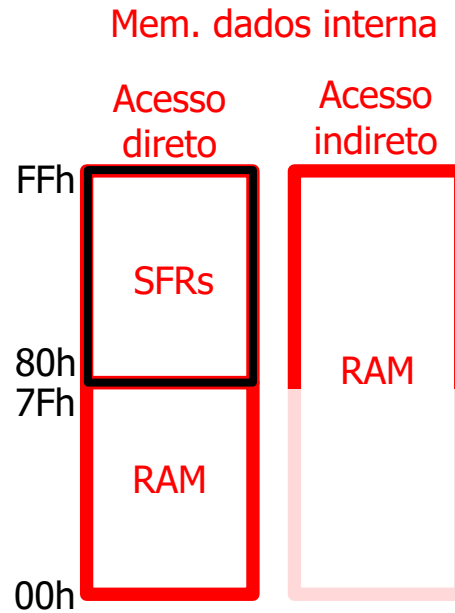




# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis
  - Mem. interna
    - Espaço de memória dos SFR
      - Não podemos declarar variáveis
        - Os registos de configuração/dados ocupam esta memória





# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis
  - Mem. interna
    - Espaço de memória dos SFR
      - Não podemos declarar variáveis
        - Os registos de configuração/dados ocupam esta memória
        - ex
          - `__sfr __at 0x80 P0;`



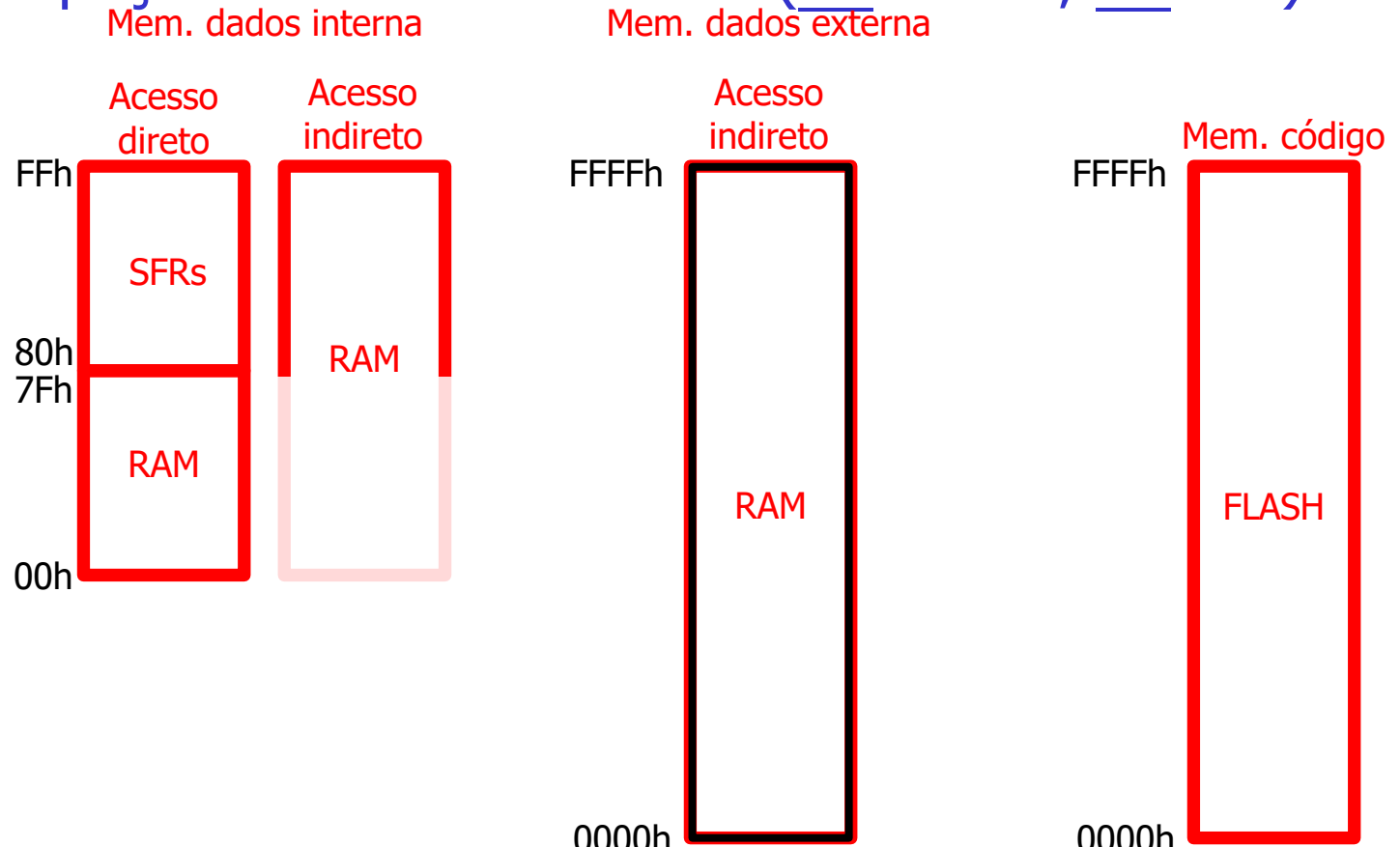
# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis

- Mem. externa

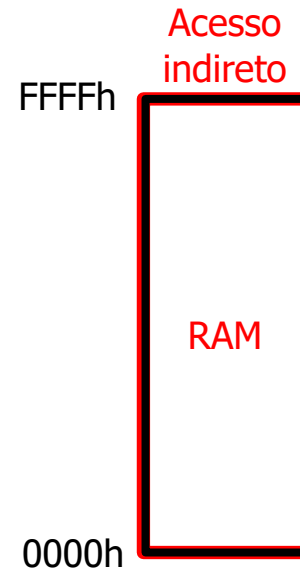
- Espaço de memória externa (`__xdata`, `__far`)





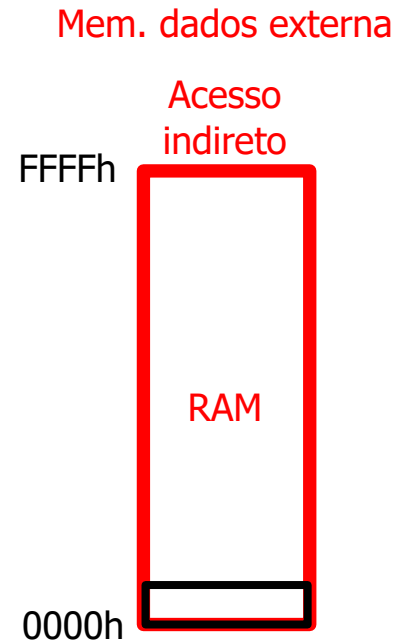
- Declaração de variáveis
  - Mem. externa
    - Espaço de memória externa (`__xdata`, `__far`)
      - Usado por defeito no modelo de memória **large**
        - `ex`
          - `__xdata unsigned char test_xdata;`

Mem. dados externa





- Declaração de variáveis
  - Mem. externa
    - Espaço de memória externa (`__pdata`)
      - Usado por defeito no modelo de memória **medium**
        - Corresponde a um bloco de 256 bytes de memória externa





# *O microcontrolador 8051*

## *programação em C*

- Declaração de variáveis
  - Mem. externa
    - Espaço de memória externa (`__pdata`)
      - Usado por defeito no modelo de memória **medium**
        - `ex`
          - `__pdata unsigned char test_pdata;`

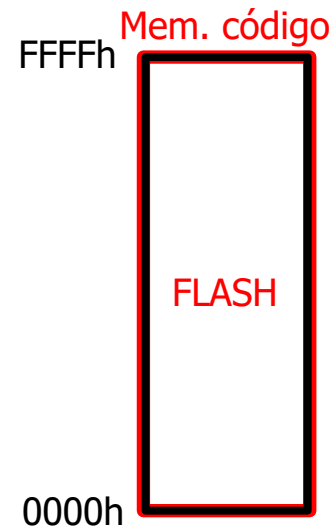




# *O microcontrolador 8051*

## *programação em C*

- Memória de código
  - Podem ser declaradas "variáveis"
    - Espaço de memória do código (`__code`)
      - Pode ser usado para armazenar constantes
        - ex
          - `__code unsigned char test_code;`





# *O microcontrolador 8051*



Embedded Systems  
Research Group



Obrigado